

Fieldstock Project Guide

1. Project Overview

Fieldstock records equipment stock and its movement among bases. Commanders can review available inventory, purchases, transfers, personnel assignments, and expenditure from one authenticated console. Every ledger entry records its actor and is visible in the audit view permitted to that role.

Assumptions: opening balances are the initial stock snapshot per base and equipment item; quantities are whole numbers in each item's configured unit; assignment, expenditure, and transfer-out movements reduce available stock; a transfer is stored once with source and destination and appears as an inbound or outbound row in base views.

Limitations: the demo catalog and opening balances are seeded examples; user provisioning is through the seed/admin process (there is deliberately no public self-registration); there is no assignment-return workflow, supplier master, approval workflow, or external identity provider. Seeded demo credentials must not be used in production. This is a project starter, not a certified operational system.

2. Stack and Architecture

- **React 19, Vite, Tailwind CSS 4, Recharts, lucide-react:** responsive browser UI, charts, and consistent icons. Visual tokens and responsive composition live in `client/src/styles.css`.
- **Node.js 20+, Express 5, Zod:** REST API with schema validation, security headers, login throttling, request logging, and centralized errors.
- **MongoDB 7+ and Mongoose 8:** MongoDB is the selected NoSQL database from the requested MERN stack. Its document model fits flexible audit snapshots and movement references; indexed event records support `base/date/equipment` filters. The trade-off is that schema and balance rules are enforced by application code rather than relational constraints.
- **JWT and bcrypt:** short-lived (8-hour) signed bearer tokens identify users; bcrypt hashes passwords. Tokens are held in browser session storage for this demo.

The UI calls the API directly. The server is authoritative for authentication, role access, base scope, request validation, and stock availability. Inventory is calculated from opening-balance snapshots and the movement ledger, not from an independently editable counter.

3. Data Models

- **Base:** name, location, timestamps.
- **Asset:** unique code, name, class, unit, description.
- **User:** name, unique email, bcrypt password hash, role, optional assigned base, active flag.
- **OpeningBalance:** one unique base/equipment pair and its starting quantity.
- **Activity:** append-only operational event with kind, actor, asset, base or transfer endpoints, quantity, event date, reference, details, old/new audit snapshots, and request IP. Login events use the same collection. A transfer is one event, so source and destination cannot be separately logged into mismatched records.

- **StockLock:** short-lived per-base/equipment locks serialize concurrent stock reductions. Locks are removed after the ledger write; expired locks can be reclaimed after process interruption.

Relationships are ObjectId references: a user may be assigned to one base, an opening balance belongs to one base and one asset, and an activity references the acting user plus the relevant asset/base records. A transfer references two bases. Indexes cover unique base/asset openings, event chronology, base activity, and transfer endpoints.

Available stock = opening stock + purchases + transfer-ins – transfer-outs – assignments – expenditures. For a date range, opening balance is computed stock before the range; closing balance applies range movements; net movement counts purchases and transfers only. Assignment and expenditure totals remain separate.

4. RBAC

All protected routes require a valid bearer JWT. Authentication reloads the active user for each request; role middleware gates writes and audit views; base-scoping middleware forces Base Commander reads to their assigned base and rejects writes naming another source/base. Base Commander transfers must originate at their assigned base. The API remains authoritative if a client manipulates navigation or request parameters.

- **Admin:** all bases, all inventory operations, audit access.
- **Base Commander:** assigned-base data and operations; may transfer stock out from that base; no cross-base source access; audit access is scoped to the assigned base.
- **Logistics Officer:** global inventory/dashboard reads and purchase/transfer operations/history; assignment and expenditure writes and the full audit endpoint are forbidden.

Public registration is omitted to prevent users from granting themselves elevated roles. Production account provisioning should be administrator-only and reviewed.

5. API and Audit Logging

Every HTTP request is logged as structured method/path/status/actor/duration metadata. Passwords, authorization headers, and request bodies are not logged. Each successful business transaction is one **Activity** insert containing operational movement and audit attribution; successful login is also recorded. The activity ledger has no update/delete endpoint. Stock-reducing requests acquire shared MongoDB locks in stable key order, recheck current stock, then write one event; insufficient stock returns 409.

Key endpoints:

Method	Endpoint	Access
POST	/api/auth/login	Public, throttled
GET	/api/auth/me	Any signed-in user
GET	/api/bases, /api/assets	Signed in; bases are role-scoped
GET	/api/dashboard, /api/inventory, /api/movements	Signed in; filters and scope applied
GET	/api/audit	Admin, Base Commander (scoped)
POST	/api/purchases, /api/transfers	Admin, Base Commander, Logistics Officer

Method	Endpoint	Access
POST	/api/assignments, /api/expenditures	Admin, Base Commander
GET	/api/health	Public health check

Filter query parameters include `baseId`, `assetType`, `startDate`, and `endDate`. GET `/api/dashboard` returns `openingBalance`, `closingBalance`, `netMovement`, `assigned`, and `expended`.

Example purchase:

```
POST /api/purchases
Authorization: Bearer <token>
Content-Type: application/json

{
  "assetId": "<asset ObjectId>",
  "baseId": "<base ObjectId>",
  "quantity": 120,
  "eventDate": "2026-09-27",
  "detail": "Supplier invoice INV-204"
}
```

Transfers use `sourceBaseId` and `destinationBaseId` instead of `baseId`. Assignment and expenditure bodies use `baseId` and are checked against available stock.

6. Setup

1. Install Node.js 20.19+ and MongoDB 7+.
2. Start MongoDB with a disposable local data directory:

```
``sh
mongod --dbpath /tmp/fieldstock-db --bind_ip 127.0.0.1
...

```

3. Copy `server/.env.example` to `server/.env`; set `MONGODB_URI` and a long random `JWT_SECRET`. Set `DEMO_PASSWORD` before seeding to override the sample password.
4. From the project root, run `npm install` and `npm run seed -w server` once. The seeder replaces data in the configured database; never aim it at production or a database with user data.
5. Run `npm run dev` from the root. Visit `<http://localhost:5173>`; API health is at `<http://localhost:4000/api/health>`.
6. Verify a production client build with `npm run build`.

`client/.env.example` documents `VITE_API_URL` for hosted deployments. The local default is `http://localhost:4000/api`.

7. Demo Credentials

All seeded accounts share `DemoPass!26` by default:

- Admin: `admin@fieldstock.demo`

- Base Commander (North Ridge): `commander@fieldstock.demo`
- Logistics Officer: `logistics@fieldstock.demo`

The seeder reads `DEMO_PASSWORD`, `DEMO_ADMIN_EMAIL`, `DEMO_COMMANDER_EMAIL`, and `DEMO_LOGISTICS_EMAIL`. These are public demo credentials; change them and disable demo seeding before hosted production use.

8. Database Dump and Deployment

`database-dump/fieldstock/` is a BSON dump of the demo database. Restore it into a local MongoDB instance with:

```
mongorestore --uri mongodb://127.0.0.1:27017/fieldstock database-dump/fieldstock
```

For Vercel, import the repository, build with `npm run build`, publish `client/dist`, and set `VITE_API_URL` to the public Render API URL ending in `/api`. For Render, use `render.yaml`; configure `MONGODB_URI` with MongoDB Atlas and `CLIENT_ORIGIN` with the Vercel origin. Render generates `JWT_SECRET`. Both platforms require account ownership and environment setup; this workspace contains no deployment credentials, so no public hosted URL has been created.

The PDF companion is `docs/PROJECT_GUIDE.pdf`. A timed recording outline for the requested walkthrough is `docs/VIDEO_WALKTHROUGH.md`; the actual video still needs to be recorded and uploaded by the project owner.